Passmngr

A CLI Password Manager Written in Rust

Personal Project Report

Author	Dario Marcone (22deeme22)
Language	Rust
Project type	Personal project: CLI tool
Repository	github.com/22deeme22/passmngr
Report date	January 2025

1. Project Overview

Passmngr is a command-line interface (CLI) password manager, written entirely in Rust. The goal was to build a simple, secure, and self-contained tool to store credentials locally, with no dependency on a third-party service or graphical interface.

This project grew out of a personal interest in computer security and the Rust programming language, which I use to learn and practise system programming best practices.

2. Features

2.1 Available Commands

The tool exposes five sub-commands:

- add – add an entry (service, login, password)
- remove – delete an existing entry
- info – display the credentials for a given service
- list – list all stored entries
- passwd – change the master password

Command	Example	Description
add	`passmngr add -s github -l user`	Add an entry (service + login)
remove	`passmngr remove github`	Delete an existing entry
info	`passmngr info github`	Show login and password for a service
list	`passmngr list`	List all stored entries
passwd	`passmngr passwd`	Change the master password

3. Security

3.1 Data Encryption

All data is encrypted using ChaCha20-Poly1305, a modern Authenticated Encryption with Associated Data (AEAD) cipher. Each write operation generates a unique random 12-byte nonce, ensuring that no two ciphertexts are ever identical, even for the same plaintext.

3.2 Key Derivation

The master password is never stored. Instead, it is passed through Argon2 (default variant) to derive a 256-bit encryption key, combined with a random 16-byte salt that is generated when the data file is first created.

The salt is stored in plaintext at the beginning of the binary file. This is standard practice: the salt does not need to be secret, only unique.

3.3 Data File Structure

The binary file stored at ~/.config/passmngr/data.bin has the following layout:

```
[16 bytes: Argon2 salt]  [12 bytes: ChaCha20 nonce]  [N bytes: encrypted data]
```

The encrypted payload is a JSON serialisation of the entries, whose integrity is verified at read time by the Poly1305 authentication tag.

3.4 Robustness

If an incorrect master password is supplied, decryption fails immediately because the authentication tag is invalid, and the program exits. No partial data is ever exposed.

4. Code Architecture

4.1 Organisation

The project intentionally fits within a single main.rs file, which is appropriate for a tool of this size. The logical structure is as follows:

- Data structures: Entry (one credential record), Cli / Commands (CLI parsing via clap)
- Utility functions: encrypt(), decrypt(), derive_key(), get_data_path()
- Entry point main(): file handling, password prompt, command dispatch

4.2 Key Dependencies

- clap: CLI argument parsing with sub-commands
- serde / serde_json: JSON serialisation of entries
- chacha20poly1305: AEAD encryption
- argon2: memory-hard key derivation
- rpassword: secure password input (no terminal echo)

5. Installation & Usage

Installation is handled by Cargo, the Rust package manager:

```
cargo install --git https://github.com/22deeme22/passmngr
```

If the passmngr command is not found after installation, make sure ~/.cargo/bin is in your PATH by adding the following line to your ~/.bashrc or ~/.zshrc:

```
export PATH="$HOME/.cargo/bin:$PATH"
```

6. Limitations & Future Work

6.1 Current Limitations

- No built-in strong password generator
- Personal-use project only: not intended for highly sensitive data

6.2 Potential Improvements

- Add a generate command to create random passwords
- Support encrypted export for backup purposes
- Add a session timeout or automatic lock after inactivity

7. Conclusion

Passmngr is a small project through which I explored Rust, basic applied cryptography (ChaCha20-Poly1305, Argon2), and CLI tool design.

The implementation includes standard security practices such as authenticated encryption, key derivation, and the use of a unique nonce for each write operation.

The source code is available on GitHub.